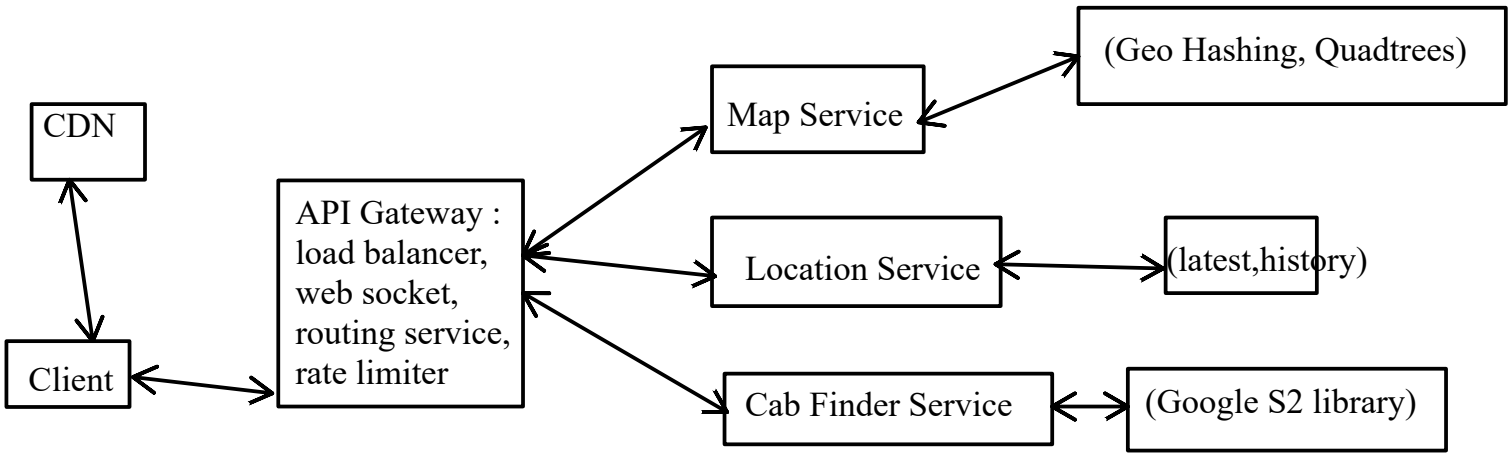
Uber
Functional requirements:
user should be able to book any cab based on his preference,
rider should be able to accept/reject any rider based on his preference,
user should be able to search for any location based on his preference,

Non-functional requirements :
Data-partitioning,
load balancing,
fault tolerance,
vertical scalability,
horizontal scalability,
low latency,
high throughput,
CAP theorem (Here we need strong consistency for booking scenario in order to avoid double booking/double payment & for search scenario we need to prioritise 99.99% uptime i.e Availability>>Consistency)

Core Entities:
user,
ride,
payment,
history

API :
POST:/cab/booking/
{
userID,
{locationID}
}
GET:/search/{locationID}

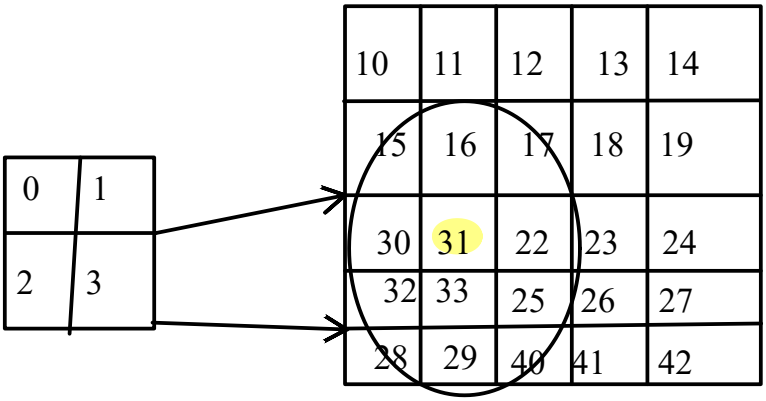
HLD :



Map Service

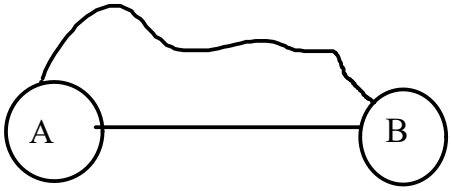
GeoHashing

(lat,long) -> ChunkID



Cab Finder Service

Google S2 Library



Consistent hashing , Gossip protocol